

Aula 9 – Paradigma Funcional

Linguagem Scala – PARTE I

1. Objetivos desta Aula

- Desafio de aprender uma linguagem de programação nova
- Conhecer uma linguagem de programação do Paradigma Funcional
- Aprender os fundamentos da Linguagem de Programação Scala.

2. Roteiro da Aula

- **PRIMEIRO**: realize os exercícios abaixo e já procure ir entendendo a linguagem Scala. NÃO VÁ DIREITO para a Lista de Exercícios!
- **SEGUNDO**: acesse os materiais da **Linguagem Scala** e **ESTUDE** estes conteúdos junto com seu grupo.
- **TERCEIRO**: Tente fazer a **Lista de Exercícios desta aula**. Se tiver dúvidas, o(a) professor(a) poderá “dar alguma dica” para ajudar, mas no primeiro momento não dará a solução. Isso para que você e seu grupo tenham a experiência de aprender uma linguagem nova “por conta” (algo que ocorrerá por muitas vezes quando você terminar a faculdade!!!).

(Obviamente, o(a) professor(a) poderá tirar suas dúvidas em um momento posterior).

3. Ambiente para Executar Scala

<https://scastie.scala-lang.org>

<https://onecompiler.com/scala>

4. Exemplos em Scala

Exemplo 1 – Fatorial de 5

```
object CalculadoraFatorial {
```

```

def calcularFatorial(numero: Int): BigInt = {
  if (numero == 0) 1
  else (1 to numero).map(BigInt.apply).product
}

def main(args: Array[String]): Unit = {
  val numero = 5 // Calculando o fatorial de 5

  if (numero < 0) {
    println("Não é possível calcular o fatorial de números negativos.")
  } else {
    val resultado = calcularFatorial(numero)
    println(s"O fatorial de $numero é: $resultado")
  }
}

```

Exemplo 2 – Cálculo da soma de dois vetores de tamanho 6

```

object SomaDeVetores {
  def somarVetores(vetor1: Array[Int], vetor2: Array[Int]): Array[Int] = {
    require(vetor1.length == vetor2.length && vetor1.length == 6, "Os vetores devem ter o mesmo tamanho (6).")
    vetor1.zip(vetor2).map { case (x, y) => x + y }
  }

  def main(args: Array[String]): Unit = {

```

```

val vetor1 = Array(1, 2, 3, 4, 5, 6)
val vetor2 = Array(6, 5, 4, 3, 2, 1)

val resultado = somarVectores(vetor1, vetor2)

println("Vetor 1: " + vetor1.mkString(", "))
println("Vetor 2: " + vetor2.mkString(", "))
println("Resultado da soma: " + resultado.mkString(", "))
}
}

```

Exemplo 3 – Multiplicação de duas matrizes de ordem 3

```

object MultiplicacaoMatrizes {
    def multiplicarMatrizes(matriz1: Array[Array[Int]], matriz2:
Array[Array[Int]]): Array[Array[Int]] = {
        require(matriz1.length == 3 && matriz1(0).length == 3, "A primeira
matriz deve ser 3x3.")
        require(matriz2.length == 3 && matriz2(0).length == 3, "A segunda
matriz deve ser 3x3.")

        val resultado = Array.ofDim[Int](3, 3)

        for (i <- 0 until 3) {
            for (j <- 0 until 3) {
                for (k <- 0 until 3) {
                    resultado(i)(j) += matriz1(i)(k) * matriz2(k)(j)
                }
            }
        }
    }
}

```

```
}  
}
```

```
    resultado  
}
```

```
def main(args: Array[String]): Unit = {  
    val matriz1 = Array(Array(1, 2, 3), Array(4, 5, 6), Array(7, 8, 9))  
    val matriz2 = Array(Array(9, 8, 7), Array(6, 5, 4), Array(3, 2, 1))  
  
    val resultado = multiplicarMatrizes(matriz1, matriz2)  
  
    println("Matriz 1:")  
    imprimirMatriz(matriz1)  
    println("Matriz 2:")  
    imprimirMatriz(matriz2)  
    println("Resultado da multiplicação:")  
    imprimirMatriz(resultado)  
}
```

```
def imprimirMatriz(matriz: Array[Array[Int]]): Unit = {  
    for (i <- 0 until 3) {  
        for (j <- 0 until 3) {  
            print(matriz(i)(j) + " ")  
        }  
        println()  
    }  
}
```

```
}  
}
```

Exemplo 4 – Calcular a posição da sub string “fofo” na string
"Gatossaoosanimaismaisfofosdaterra".

```
object BuscaSubstring {  
  def encontrarSubstring(stringPrincipal: String, subString: String):  
    Option[Int] = {  
    val index = stringPrincipal.indexOf(subString)  
    if (index != -1) Some(index)  
    else None  
  }  
}
```

```
def main(args: Array[String]): Unit = {  
  val stringPrincipal = "Gatossaoosanimaismaisfofosdaterra"  
  val subString = "fofo"  
  
  encontrarSubstring(stringPrincipal, subString) match {  
    case Some(posicao) => println(s"A substring '$subString' foi encontrada  
na posição $posicao.")  
    case None => println(s"A substring '$subString' não foi encontrada na  
string principal.")  
  }  
}
```